TD N°3 - Redirections et processus

Objectifs :

- **se familiariser** avec les **redirections** ou le fait de pouvoir utiliser le résultat d'une commande pour la commande suivante (**tube** ou **pipe**).
- **comprendre** les notions d'**entrée/sortie** standard, **redirections**.
- **approfondir** quelques commandes de **filtrage** de fichiers, et leurs utilisations avec les **expressions régulières**.
- **découvrir** la notion de **processus**.

RAPPEL : La commande **man [commande]** (pour *manual*) donne une aide pour chaque commande. Une fois que la page manuel est affichée, les touches de déplacement permettent de naviguer dedans. Pour quitter et revenir à la ligne de commande, il suffit de taper **q**. Il y a beaucoup de subtilités que l'on passe ici sous silence, n'hésitez pas à vous servir de `man` pour en apprendre plus !

Rappel : Entrées/Sorties, Redirections, et Tubes

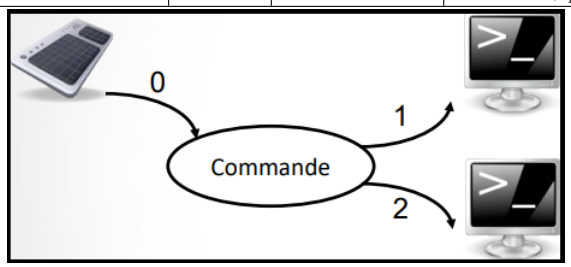
Entrées et Sorties

Une *commande* (un *programme* ou un *processus*) consiste à traiter des données, et à renvoyer des données transformées. Lors de son lancement, une commande dispose par défaut de trois flux standards :

- l'**entrée standard** (`stdin`) ;
- la **sortie standard** (`stdout`), pour les *messages normaux* relevant de la transformation d'informations ;
- la **sortie d'erreur standard** (`stderr`), pour les *messages d'erreur* (en cas de problème).

Pour résumer, une commande possède trois voies d'interaction avec l'extérieur :

Description	Nom	Descripteur	Périphérique
Entrée standard	stdin	0	terminal, par défaut
Sortie standard	stdout	1	terminal, par défaut
Sortie standard d'erreur	stderr	2	terminal, par défaut



Redirections

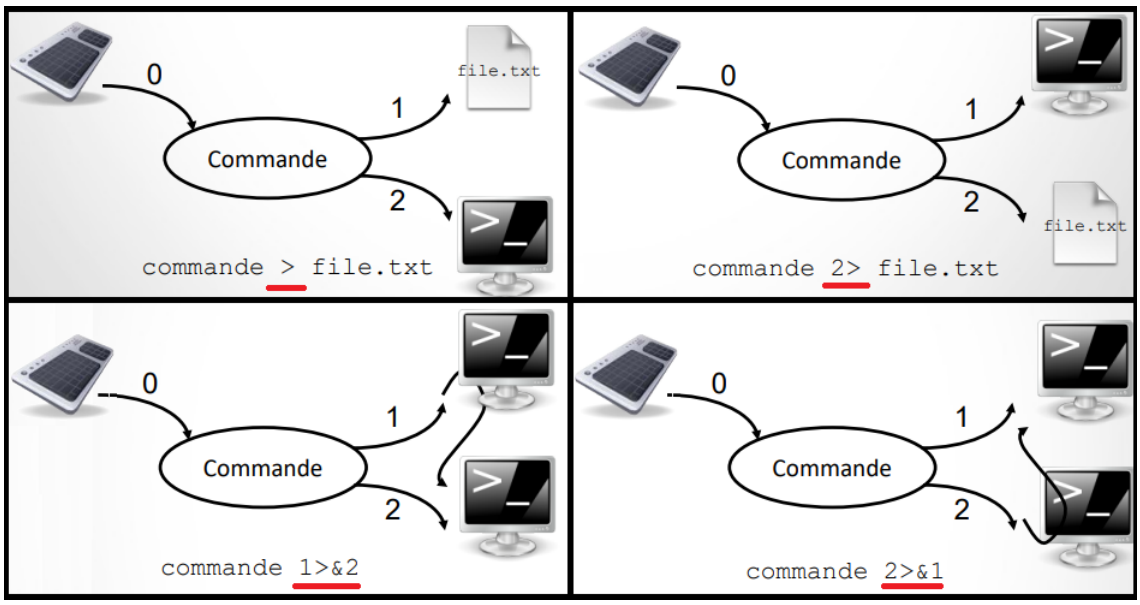
On peut *rediriger* séparément chacune des trois entrées/sorties/erreurs standards d'une commande. Cela signifie qu'une commande pourra :

- lire les données à traiter à partir d'un fichier et non du clavier de l'utilisateur,
- écrire les résultats ou erreurs dans un fichier et non à l'écran.

Une redirection associe l'entrée ou une sortie d'une commande à un fichier ou à un autre descripteur, grâce à un ou des caractères spéciaux. Un tube (*pipe*), quant à lui, connecte deux commandes entre elles (voir plus loin).

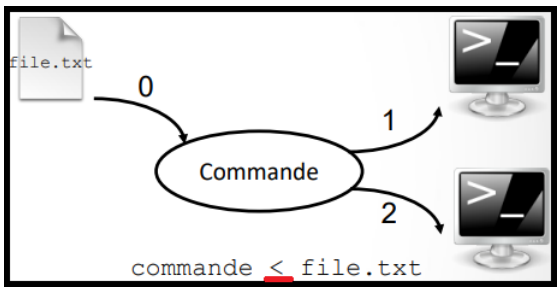
1. Redirection des sorties

- **rediriger** la sortie d'une commande vers un fichier avec les caractères : `>` ou `1>`
  En **écrasant le contenu** du fichier ou en le **créant** s'il n'existe pas.
- **ajouter** la sortie d'une commande à la **fin** d'un fichier avec le caractère : `>>`
  En **ajoutant** le résultat **à la fin** du contenu du fichier, **sans effacer** ce qui précède.
- **rediriger** la sortie standard d'erreur avec le caractère : `2>`
  En écrasant le contenu du fichier ou en le créant s'il n'existe pas.
- **ajouter** la sortie standard d'erreur à la **fin** d'un fichier avec le caractère : `2>>`
  En **ajoutant** le résultat **à la fin** du contenu du fichier, **sans effacer** ce qui précède (et en le **créant** s'il n'existe pas).
- rediriger **un canal vers un autre canal** avec le caractère : `n>&m`
  En spécifiant quel est le numéro de canal `n` que l'on redirige à gauche vers le numéro `m` de canal à droite (**sans espace** entre les caractères, sous peine d'erreur de syntaxe).
  - rediriger la sortie standard d'erreur vers la sortie standard avec le caractère : `2>&1`
  - rediriger la sortie standard vers la sortie standard d'erreur avec le caractère : `1>&2`



2. Redirection de l'entrée

- rediriger l'entrée standard d'une commande avec le caractère : `<`
  La commande lira alors dans le fichier au lieu du clavier.



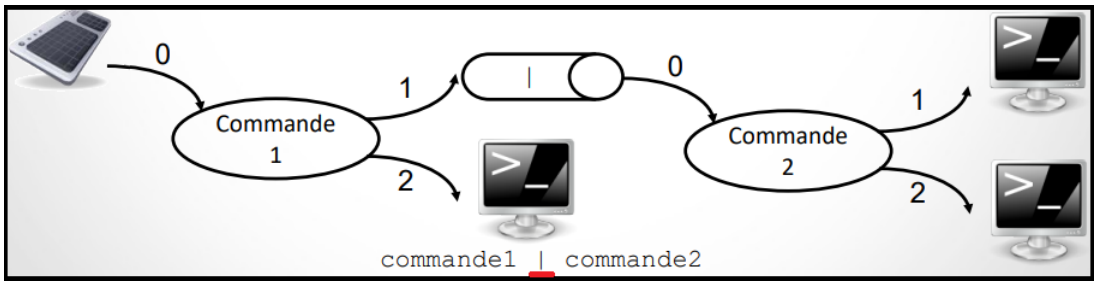
Attention : il est possible de rassembler ces opérations de redirections dans une même commande (e.g. `wc -l < toto.txt 1> tata.out 2> titi.err`)
Les redirections étant traitées de gauche à droite, l'**ordre** des redirections est **important**. Par exemple :

- `commande > tout.log 2>&1` : `stdout` est d'abord envoyé dans *tout.log*, puis `stderr` est envoyé vers cette même destination (*tout.log*) ;

- `commande 2>&1 > tout.log` : `stderr` est d'abord associé à la destination actuelle de `stdout` (le terminal), puis seule `stdout` est redirigée vers *tout.log* ; `stderr` continue donc de s'afficher à l'écran.

Pipe ou tube

Un pipe (|) connecte directement la sortie standard d'une commande sur l'entrée standard d'une autre commande.
Ainsi, on peut enchaîner plusieurs commandes les unes à la suite des autres.



Attention : par défaut, un tube ne transmet que la **sortie standard** (`stdout`), et non la sortie d'erreur (`stderr`).
Pour transmettre également `stderr` au travers du tube, il faut d'abord la rediriger vers `stdout` :
`commande 2>&1 | autre_commande`

Rappel : gestion des processus

Définition

Un processus correspond à l'exécution d'un programme ou d'une commande Linux. Un processus possède des caractéristiques qui permettent au système de l'identifier.
Parmi ces caractéristiques, on trouve : son identifiant (process ID ou PID), l'identifiant du processus qui lui a donné naissance, son processus parent, (parent PID ou PPID), son répertoire de travail, sa priorité, son temps d'exécution, ...

L'interpréteur de commandes `shell` est un programme et il devient un processus lorsqu'il est exécuté.
Pour exécuter une commande **externe**, le `shell` crée généralement un processus enfant, puis y charge le programme demandé. Les commandes **internes** du `shell` (comme `cd`, `export`, `alias`, `jobs`, `fg`) sont, elles, normalement exécutées directement par le `shell`, sans création d'un nouveau processus.

Gestion des processus

La commande `ps` affiche un instantané des processus démarrés dans le système.

Gestion des processus en temps réel

La commande `top` surveille et affiche dynamiquement, en continu, les processus démarrés dans le système en temps réel.
Cette commande affiche également le pourcentage du temps processeur (%CPU) et de mémoire physique (%MEM) utilisée par le processus.

Contrôle de l'exécution des processus

La commande `kill` permet d'envoyer différents types de signaux à un processus dont on connaît l'identifiant. Malgré son nom, elle ne sert pas seulement à "tuer" un processus.
De nombreux signaux peuvent être envoyés par cette commande, pour les lister : commande `kill -l`. Vous trouverez la signification dans le manuel : `man 7 signal`.
Les signaux les plus courants sont : SIGSTOP (19) pour suspendre de force un processus, SIGCONT (18) pour reprendre son exécution, SIGTERM (15) pour signifier au processus qu'il doit se terminer, SIGKILL (9) pour tuer un processus.

Contrôle des tâches

Dans un shell `bash`, on peut lancer et contrôler plusieurs tâches, appelées *jobs*. Un job peut correspondre à une commande simple ou à une chaîne de plusieurs processus.
Par défaut, un job est exécuté au premier plan. Le groupe de processus au premier plan peut alors recevoir les données saisies au terminal. Pour le suspendre, il faut utiliser `Ctrl+Z`.
Pour démarrer un processus en arrière-plan, il faut utiliser le signe `&`.

Les commandes pour manipuler les jobs sont les suivantes :

- démarrer un processus en arrière-plan avec le caractère `&`.
- lister tous les jobs actifs `jobs`.
- ramener un job en avant-plan avec `fg %numjob`.
- suspendre un job et le faire passer à un état arrêté avec `Ctrl+Z`.
- reprendre en arrière-plan un job suspendu avec `bg %numjob`.
- envoyer par défaut `SIGTERM` à un job avec `kill %numjob`.

Exercices

Exercice 1

Créez le fichier *file.txt* à l'aide d'une redirection au lieu de le faire avec un éditeur de texte.

Exercice 2

En une seule commande, listez le contenu de ces trois répertoires dans un unique fichier **results.txt** :

- */usr/bin/* (qui devrait exister)
- */var/lib/apt* (qui devrait exister)
- */etc/iut-RS* (qui ne devrait pas exister)

Le fichier **results.txt** devra contenir uniquement la sortie standard produite par `ls -l` ; les messages d'erreur devront être enregistrés séparément dans **errors.log**.

Donnez la commande pour réaliser cette opération.

Exercice 3

1. Concaténez les fichiers */etc/passwd* et */etc/group* dans le fichier *comptes*, dans un répertoire personnel sous */tmp*.
2. Cherchez tous les fichiers de nom "`passwd`" depuis le répertoire */*.
   Sauvegardez le résultat de votre commande (sans les messages d'erreurs) dans le fichier *listepasswd*.
   Tous les messages d'erreur doivent être redirigés dans le fichier spécial "poubelle" */dev/null*.
3. Triez le fichier */etc/passwd* par ordre alphabétique et enregistrez le résultat dans le fichier *TriPasswd*.
4. Effectuez un tri numérique des identifiants utilisateurs `UID` du fichier */etc/passwd*, et enregistrez le résultat à la suite du fichier *TriPasswd* de la question précédente.

Exercice 4 : Gestion des processus

Principes de base des processus

1. Sur votre machine Linux, de nombreux processus sont exécutés en même temps.
   Cependant, une information peut identifier de façon unique un processus.
   Comment est-elle appelée ?

2. Lorsque votre système démarre, il lance le tout premier processus de votre instance.
   Quel est le PID de ce premier processus, et quel programme assure généralement ce rôle sur les distributions Linux utilisées en TP ?

Commandes de processus

1. Identifiez tous les processus que vous possédez sur la machine.
2. Identifiez la commande qui affiche tous les processus qui tournent sur la machine.
3. Quelle commande affiche les processus sous forme d'arbre ?

Processus en arrière-plan et en avant-plan

1. Quelle syntaxe est utilisée pour exécuter un processus en arrière-plan ?
2. Quel est le terme qui décrit un processus qui a été lancé dans un shell terminal en arrière-plan ? Et, quelle est la commande qui permet d'afficher ces processus lancés en arrière-plan ?
3. Vous avez exécuté la commande `sleep` en arrière-plan, mais vous voulez que votre processus soit exécuté en avant-plan.
   ```
   $ sleep 100 &
   [1] 3180
   $ jobs
   [1]+ Running
   $
   ```
   Quelle commande exécuteriez-vous ?
4. Votre processus est maintenant exécuté au premier plan.
   Quel raccourci permet de suspendre le job au premier plan sans le terminer ?
5. Votre job est maintenant suspendu.
   ```
   $ fg %1
   sleep 100
   ^Z
   [1]+ Stopped sleep 100
   $ jobs
   [1]+ Stopped sleep 100
   ```
   Comment reprendre l'exécution en arrière-plan ?
6. Ramenez le job au premier plan avec `fg %1`. Quel raccourci permet alors de lui envoyer une demande d'interruption provoquant généralement sa terminaison (à distinguer de `Ctrl+Z`, qui ne fait que le suspendre) ?

Signaux et processus

1. Comment définiriez-vous un signal en matière de processus ?
2. Quel est le signal utilisé pour arrêter en douceur un processus ?
3. D'autre part, disons que vous voulez tuer un processus immédiatement, quel signal utiliseriez-vous pour cela ?
   Quel est le numéro de ce signal ?

Commandes avancées sur les processus

1. Quelle commande est utilisée pour lister tous les processus en fonction d'un motif spécifique ?
   Lancez en arrière-plan un processus de test (par exemple une simple temporisation), puis utilisez cette commande pour identifier son PID.

2. Quelle commande est utilisée afin d'exécuter un processus avec un niveau de priorité personnalisé ?
3. Quel est le niveau de `nice` par défaut lors de la création de processus sous Linux ?

Contrôle de processus

1. Quelle commande peut être utilisée afin de surveiller les processus en temps réel ?
2. Quelle option peut être utilisée afin d'exécuter la commande `top` pour un nombre personnalisé d'itérations ?